# 11
# Understanding Deep Agents

In this chapter, we dive into Deep Agents. Deep Agents are agents that are capable, useful, and designed to perform a variety of tasks, specifically long-horizon tasks that require sustained reasoning and action over multiple steps. Coding agents such as Claude Code, Cursor CLI, and the LangChain Deep Agents fall into this category. These systems are not limited to single-step tool usage; they operate across extended workflows, making decisions, adapting their plans, and acting iteratively until a broader objective is achieved.

We begin by taxonomizing Deep Agents, establishing what qualifies an agent as "deep" and identifying the defining characteristics that distinguish them from simpler agent patterns. From there, we define at a high level what makes an agent a deep agent and examine the properties that enable long-horizon execution.

This provides a practical glimpse into how state-of-the-art coding agents such as Claude Code are implemented, grounding our understanding in real engineering.

The following topics will be covered in this chapter:

- Agent taxonomy in production systems
- What makes an agent deep

## Agent taxonomy in production systems

Before discussing Deep Agents, we first need to establish a taxonomy and clarify what kinds of agents are currently used in production across the industry. I will simplify this and describe how I view the landscape.

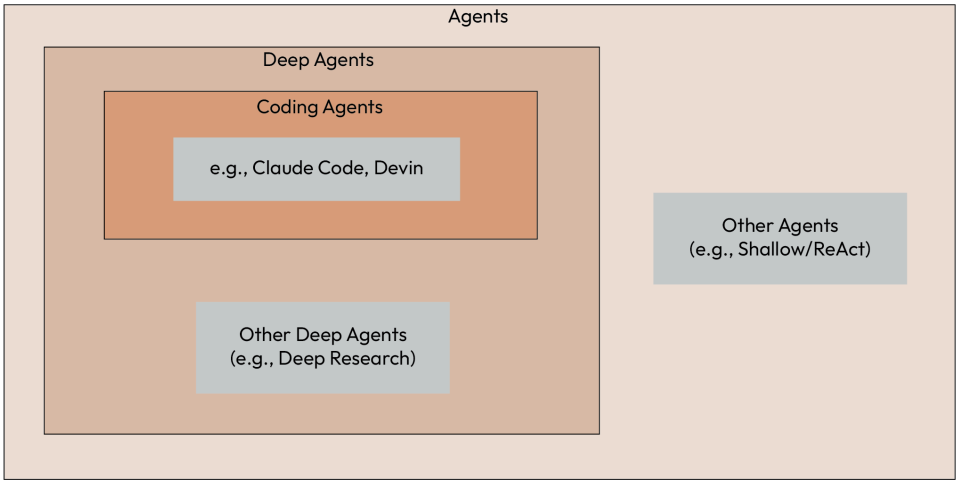


*Figure 11.1 – Agent taxonomy showing coding agents within Deep Agents*

At the broadest level, we have the domain of agents. This domain includes all types of agents currently deployed in real systems. These can be fully autonomous agents or agentic applications. A common example is a hybrid RAG architecture in which an LLM decides which step to execute next.

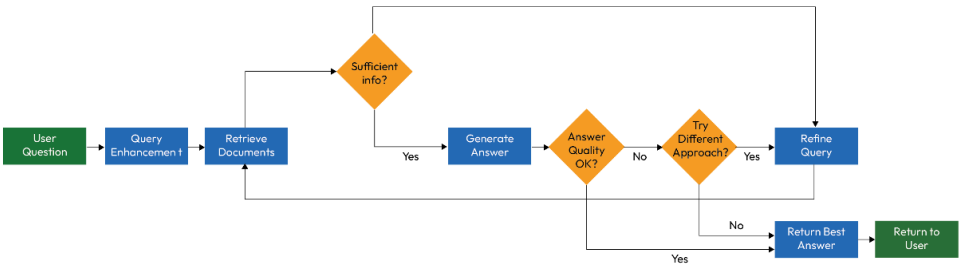


*Figure 11.2 – Hybrid RAG workflow orchestrated by an LLM*

The model may decide whether to perform a search, whether to rephrase a query before retrieval, or whether to execute another operation. The LLM is effectively orchestrating the next action in the predefined workflow.

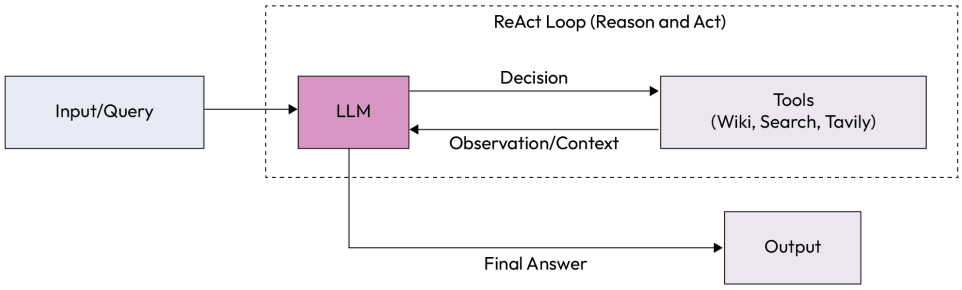


*Figure 11.3 – ReAct loop connecting the LLM to external tools*

Within this umbrella, we find the ReAct (Reason and Act) agent. In a ReAct agent, an LLM decides whether to use a tool. If a tool is selected, the system executes it and returns the result as an observation. The LLM then decides whether it has sufficient information to return a final response. If not, it continues the loop: selecting another tool, receiving another observation, and iterating until it reaches an answer.

This ReAct loop is what initiated the broader agent paradigm. The majority of modern agent architectures trace back to this algorithmic structure. The approach was introduced in the paper "*ReAct: Synergizing Reasoning and Acting in Language Models*" (`https://arxiv.org/abs/2210.03629`).

## Shallow agents and their limitations

In this taxonomy, I am categorizing the ReAct agent as a shallow agent. The reason is not that it lacks utility, but that it lacks the capability to go deep. Consider a task that requires thorough and extended research. Even if the agent has access to a search tool, a wiki tool, or a Tavily tool, it is unlikely to perform sufficiently deep research. The limitation is rooted both in architectural design and in the realities of modern LLMs.

This agent relies on function calling. At each decision point, the LLM makes a function call to select the tool to execute. There is nothing inherently wrong with function calling, although it has trade-offs. The core limitation, however, is the context window.

Every iteration in this loop adds more information to the context. The LLM decides, a tool is executed, the result is injected back into the prompt, and the cycle repeats. With one or two iterations, this remains manageable. As the number of iterations increases, the context grows larger and larger.

This growth leads to context rot. We begin to see context confusion, context contradiction, and context pollution. Over time, performance degrades and the agent may go off the rails.

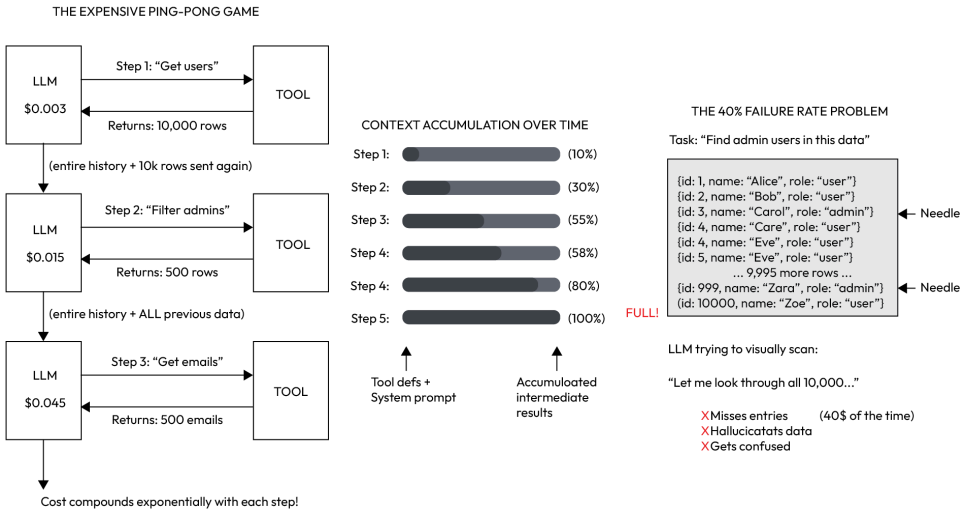


*Figure 11.4 – Context accumulation in iterative tool-calling workflows*

This limitation becomes critical when dealing with complex, long-running tasks, such as implementing a feature, conducting deep research on a topic, or performing a multi-stage workflow that requires continuous reasoning and information gathering. These are fundamentally different from simple requests, such as booking a flight.

In addition to degraded quality, the cost increases. Each LLM call becomes heavier because it includes more tokens. More tokens increase latency and expense. While the ReAct architecture is foundational and powerful, it is not well-suited for complex, long-horizon tasks.

That said, shallow agents are highly effective for tasks that require only a few iterations. Many production systems use this architecture successfully. In numerous real-world use cases, this pattern is sufficient and does not require additional complexity.

### Deep Agents

Returning to the taxonomy, we now move from shallow agents to Deep Agents.

Deep Agents are designed to perform long-horizon tasks. These tasks are complex, require many iterations, and involve significant reasoning and processing. Deep Agents are long-running systems. They may run for minutes, hours, or even days. They can pause execution, request user input, and resume after receiving that input.

Deep research agents are a common example. Systems such as Perplexity expose deep research features that trigger a deep agent run. Many vendors now provide similar research capabilities. Claude Code includes a research option. ChatGPT includes a research option. Each of these triggers a deep agent implementation, though the internal design differs across vendors.

There are also open source deep research agents. GPT Researcher (`https://github.com/assafelovic/gpt-researcher`) is one of the most popular open source projects in this category. I have created a course for this. You can find it here:

`https://www.udemy.com/course/langchain/?srsltid=AfmBOopb6uyyf5oNvX1gvDus-Nqdln5B-bwZtJAW1NLsv7z7ggPJLX9A`.

## Coding agents as Deep Agents

Another major subset of Deep Agents is **coding agents**. Examples include Claude Code, Devin, Cursor, and Gemini CLI. Coding agents are one of the strongest demonstrations of how Deep Agents can be effective and widely adopted in the industry. Millions of developers use them.

As you can see, for this book, I have mainly focused on Claude Code, which, at the time of writing this book, is one of the most popular coding agents. Claude Code is highly capable and can

handle a wide range of long-running tasks that require depth and scale.

We can ask such an agent to implement an application or a feature. It does not merely generate code. It can run tests, execute the application, open a browser, take screenshots, and perform many of the same steps a software engineer would perform manually.

Coding agents are, therefore, a specialized subset of Deep Agents, tailored specifically for software development workflows.

## Innovation at the application layer

Deep Agents currently drive much of the innovation in this space. LLMs continue to improve, but those improvements are gradual. Reasoning quality increases. Capabilities expand. However, the progress is incremental rather than exponential.

At the same time, the application layer built on top of LLMs is advancing rapidly. By layering abstractions on top of agents and composing agents within systems, we can build highly capable machines that automate complex human reasoning tasks. This application layer is often referred to as an Agent Harness.

---

NOTE

An **Agent Harness** is a ready-to-use environment for building AI agents. It sits on top of the tools and systems that run agents, and comes pre-configured with everything you need: default prompts, tool handling, planning, file access, and more. Frameworks give you raw building blocks. Runtimes manage execution. A harness does the heavy lifting so you can start building without setting everything up from scratch.

---

Five years ago, the idea that an AI system could create a functional and visually polished application from scratch without human intervention would have seemed unrealistic. Today, this capability exists.

The key driver is not only the improvement of the base models, but also how we, as developers, harness them (harness engineering). The innovation is happening in the application layer, in how we implement and orchestrate Deep Agents.

## What makes an agent deep?

There is no strict formal definition of a deep agent. In practice, I consider an agent deep if it can perform complex, long-running tasks with quality and reliability.

To achieve this, Deep Agents require capabilities that address context bloat and accumulation. The core challenge becomes context engineering and smart context management. Without it, long-horizon execution breaks down.

Most Deep Agents implement several common ideas:

- **A planning tool**: The agent plans what it is going to do before executing steps
- **Sub-agent capabilities**: The system spawns specialized workers that operate in isolated contexts. This allows concurrent execution without bloating a single shared context.
- **A filesystem for intermediate state**: Intermediate results and shared state are written to disk rather than retained entirely in the LLM context. This prevents uncontrolled context growth.
- **A large system prompt**: Deep Agents typically rely on a comprehensive system prompt that encodes instructions, constraints, and operational guidance

*If we examine modern deep agent implementations, we typically see these four ideas reflected in some form. In the next section, we will dive into each of these components and examine concrete implementations.*

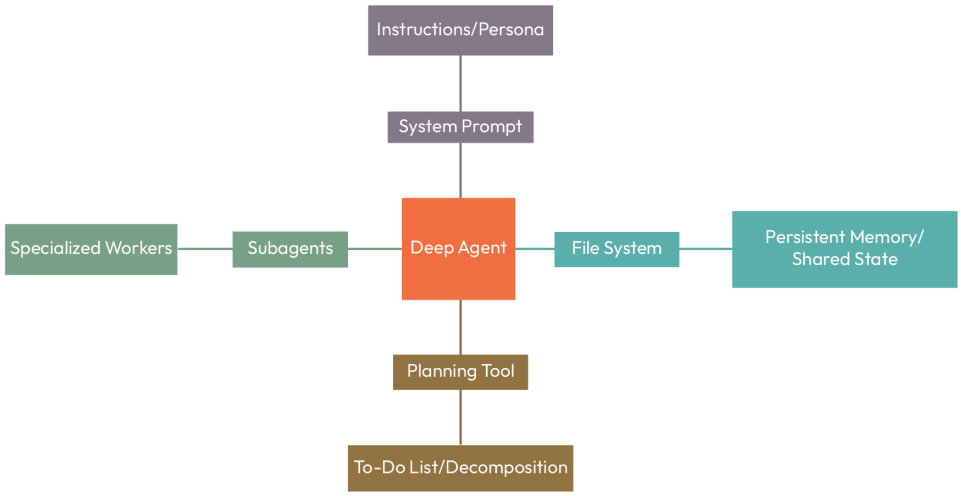


*Figure 11.5 – Core components of a deep agent architecture. Adapted from: LangChain Blog, Deep Agents, https://blog.langchain.com/deep-agents/*

## Planning tool in Deep Agents

Although "Deep Agents" is a general term, the LangChain team articulated it clearly after analyzing multiple deep agent implementations. Every deep agent includes a planning tool.

When we run the tool, we see a list of tasks: some are already completed, one may currently be in progress, and others are queued for execution. What we observe is not implicit planning through chain-of-thought reasoning inside the model. Instead, Deep Agents rely on explicit planning tools.

This planning mechanism is usually implemented as a **to-do list** in Markdown format. Between execution steps, the agent actively reviews and updates this plan. The plan is dynamic. Tasks are marked as *pending*, *in progress*, or *completed*. If a task fails, the agent does not blindly retry it as in the original ReAct agent loop. Instead, the planning tool helps steer execution in a more controlled manner.

The plan is continuously updated, and users can influence the task list. In Claude Code, the planning tool itself is internal and its implementation is not directly accessible. In practice, `TodoWrite` and `TodoRead` actions are invoked. The `TodoWrite` tool creates and updates the to-do list, taking a `{ todos : Todo[]}` parameter. The `TodoRead` tool reads the current list. This call updates the to-do list, reflecting changes in the execution plan. Even though the internal reasoning is not directly visible, actions such as `TodoWrite` and `TodoRead` provide insight into how the agent dynamically adjusts its plan while working through a task.

The deep agent continuously updates this to-do list, which improves reliability and increases the likelihood of successfully completing complex tasks. Conceptually, this is intuitive. When handling complex work, we break it down into smaller tasks and track progress over time. The planning tool formalizes that same pattern inside the agent architecture.

## Subagents and hierarchical delegation

We discussed the planning tool, or the to-do list tool. Another defining characteristic of Deep Agents is the subagent capability.

Deep Agents employ subagents to enable hierarchical delegation. The main agent can spawn new instances of itself, but those instances are specialized for focused tasks. Each subagent operates with its own system prompt, its own description, and its own set of tools, as shown in the following diagram:

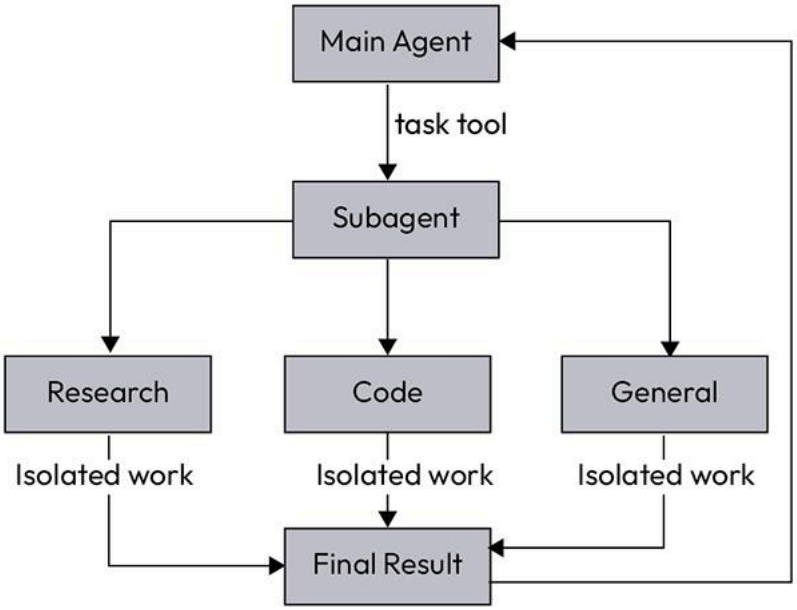


*Figure 11.6 – Hierarchical delegation using subagents*

This design mirrors how delegation works in real life. When delegating a task, it is not enough to assign the work. The delegated party must have the correct skills and the correct tools. It is also necessary to communicate clearly what needs to be done.

Consider a practical example. Suppose there is a task that requires specific craftsmanship skills, such as fixing a structural issue in a house. If the required expertise is missing, the task must be delegated to someone who has the appropriate skills and tools. That person arrives with their own equipment, performs the work independently, and delivers the final result. The details of the intermediate steps remain isolated from the original requester. What matters is the final outcome.

This is a useful analogy for subagents. The main agent delegates a task by providing a clear description. The subagent operates with its own tools and internal instructions, executes the work independently, and returns the result. During execution, the subagent works in isolation. The main agent does not see every intermediate reasoning step. It only receives the final output.

From a systems perspective, this means subagents run in their own context window. They do not pollute the main agent's context. Each subagent can have a different system prompt and a different set of tools tailored to its specialization. This enables scaling and improves quality by assigning focused responsibilities to specialized agents.

Internally, subagents execute their own tool-calling loops, including their own ReAct-style cycles. When finished, they return only the final response, not the full chain of intermediate observations. This delegation pattern achieves context isolation, prevents specialized work from bloating the main agent's context, and allows parallel execution of tasks. As a result, both efficiency and depth improve.

In practice, we can observe this pattern in systems such as Claude Code. Consider the following example:

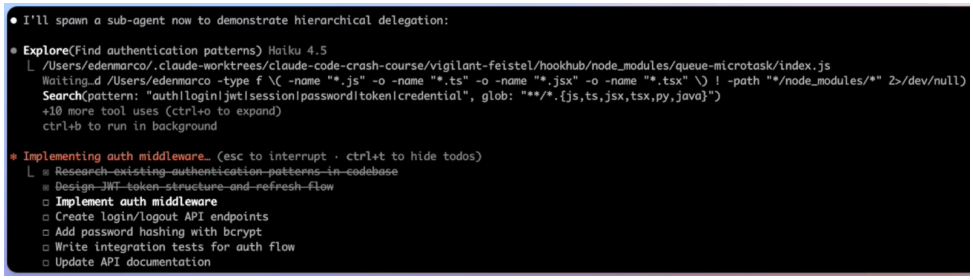


*Figure 11.7 – Claude Code spawning a subagent to explore authentication patterns*

In this example, Claude Code spawns an exploration agent to search for authentication patterns. That exploration agent runs concurrently, performs its own searches, and operates independently while the main agent continues its execution. This demonstrates built-in support for subagents and hierarchical delegation.

Up to this point, the discussion has focused on concepts and architectural reasoning rather than implementation details. The goal here is to make the conceptual structure clear and to connect these capabilities to the tools used in everyday workflows.

This builds directly on the discussion in **Chapter 7**, where we examined how subagents prevent context bloat, enforce context isolation, and ultimately lead to better results.

## Filesystem access in Deep Agents

Another key characteristic of Deep Agents is access to a filesystem. Deep Agents are equipped with tools to search for files, read files, update files, delete files, and generally maintain full control over a filesystem. This capability plays a central role in managing context for long-horizon tasks.

In Claude Code, this manifests through a set of file operation tools.

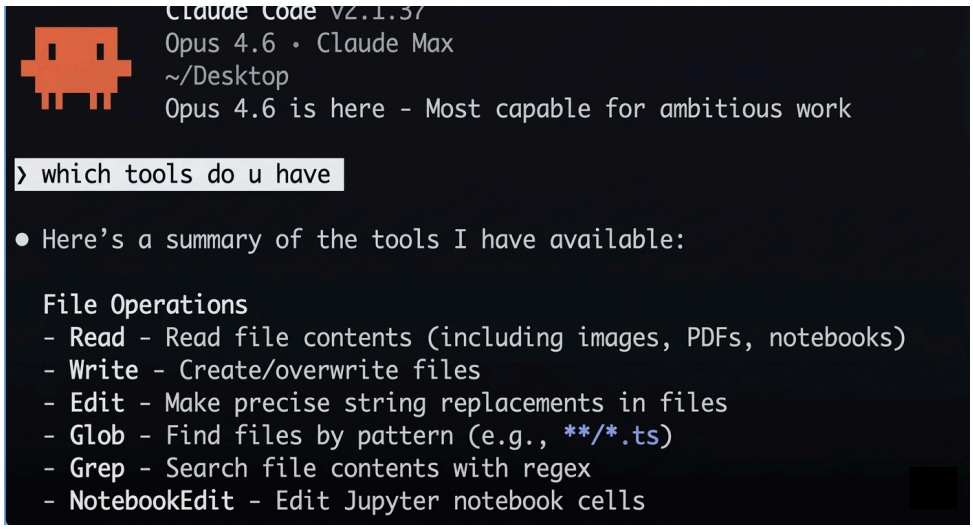


*Figure 11.8 – Claude Code file operation tools available to the agent*

As you can see, there is a `Read` tool to read file content, a `Write` tool to create or overwrite files, and an `Edit` tool to perform precise string replacements. In addition, Claude Code provides `Glob` and `Grep` tools to search for files and search within files.

If we examine LangChain's Deep Agents documentation, we see a similar filesystem interface: `ls`, `read_file`, `write_file`, `edit_file`, `glob`, and `grep`. Deep Agents expose an interface rather than a specific implementation. This means the underlying storage can vary. For exam-

ple, the filesystem could be backed by Firestore on Google Cloud or DynamoDB on AWS. The interface remains the same, while the implementation is flexible.

## Why filesystems matter for long-horizon tasks

As discussed earlier, as a conversation grows, the context window grows with it. This leads to a range of problems that ultimately result in context rot. Context rot produces degraded responses from the LLM. The degradation may stem from context contradictions, context confusion, or excessive noise in the prompt.

*In one of LangChain's blog posts, there is an illustration that captures the core challenge of context engineering for agents. The diagram shown here is inspired by that explanation.*

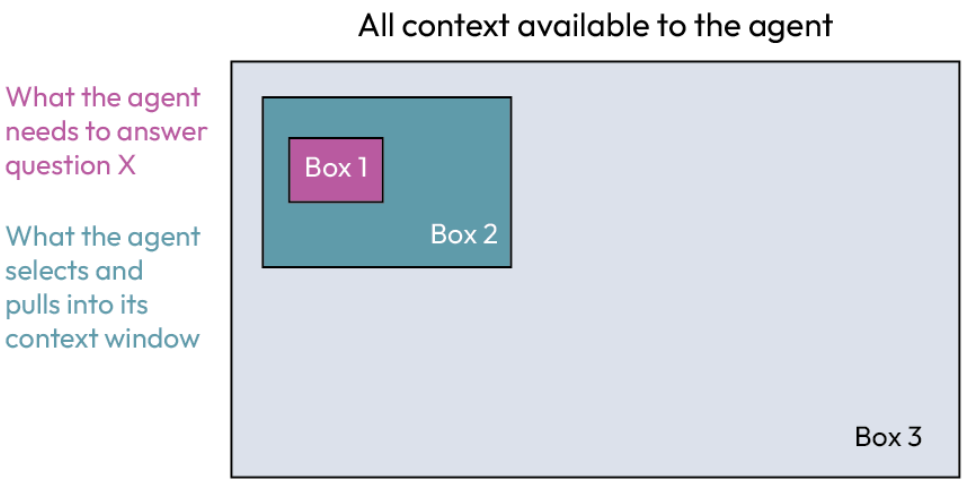


*Figure 11.9 – Available context vs. selected context vs. required information. Source: LangChain Blog, How agents can use filesystems for context engineering, https://blog.langchain.com/how-agents-can-use-filesystems-for-context-engineering/*

Imagine a large blue rectangle (Box 1) representing all available context: code bases, documents, web search results, files, and databases. This space is potentially large and messy.

Within that, a Box 2 represents the subset of information the agent selects and pulls into the context window for a given step. This is what the agent chooses to read or search. Finally, Box 3 represents the information the agent actually needs to complete the task.

Whenever the red circle does not properly align with the green circle, several failure modes emerge:

- **Under-retrieval**: The agent retrieves only part of Box 2 and misses the necessary information.
- **Over-retrieval**: **Box 2** becomes too large, pulling in excessive noise and diluting the signal.
- **Misaligned retrieval**: The agent searches in the wrong place, and **Box 1** does not overlap with the **Box 2** at all.
- **Context window limits**: The purple box is finite. It cannot contain everything, and if it becomes too large, performance degrades.

The goal of context engineering is to make the red circle as small as possible while fully covering the green circle. This selection process can occur at almost every iteration of the agent. Each step requires optimizing what to include in the context window.

For this reason, how information is structured, retrieved, and prioritized often matters more than the prompt itself. The agent's quality is bound by whether the right information is present in the context window. Even with a strong reasoning model, incorrect or incomplete context can lead to incorrect results or an inability to complete the task.

## The filesystem as a context engine

This is where the deep agent's filesystem becomes critical. The filesystem acts as the engine that enables the agent to reach the correct context. Conceptually, the entire filesystem corresponds to the blue rectangle of available information:

1. First, the filesystem allows the agent to write context to persistent storage. Temporary results, intermediate outputs, or retrieved information can be written to files. Instead of bloating the context window, these artifacts are stored externally.
2. Second, the filesystem provides mechanisms for selecting relevant context. Tools such as `glob` allow the agent to find files by pattern, and `grep` allows it to search file contents using regular expressions. These tools enable the agent to retrieve precisely what is needed for the next step.

In this way, the filesystem implements two core principles of context engineering (**Chapter 1**): writing context to persistent storage and selectively retrieving relevant context. It is the mechanism that enables both, and it is fundamental to how Deep Agents manage long-horizon tasks without suffering from context bloat.

## System prompt

In AI engineering, it has become somewhat of a cliché to state that "*system prompts are important*." However, observing the actual practices of industry leaders reveals just how critical this component is. We see system prompts that span hundreds of lines, heavily augmented with massive, injected tool descriptions.

Companies dedicate immense engineering resources to iteratively curating, refining, and updating these prompts as LLMs evolve. For a deep agent, the system prompt is the fundamental architecture of its reasoning, identity, and boundaries.

A highly effective system prompt leverages what state-of-the-art LLMs naturally excel at: pattern recognition and applying general rules to specific, novel situations. A well-crafted system prompt achieves the following:

- **Establishes clear identity and scope**: By clearly defining what the agent is (e.g., "customer support for basic orders") and what it is not (e.g., "not sales or marketing"), the prompt immediately establishes operational boundaries
- **Empowers rather than constrains**: Instead of dictating exactly which tool to use at what time, a great prompt establishes the end goal (e.g., "efficient resolution") and empowers the agent to select the appropriate tools autonomously.
- **Provides a reasoning framework, not a flowchart**: Instead of rigid "If/Then" branching logic, the prompt gives a repeatable framework (e.g., 1. Identify the core issue, 2. Gather context, 3. Provide resolution, 4. Confirm satisfaction). This teaches the model how to approach problems, allowing it to succeed even when it encounters entirely novel situations.
- **Utilizes heuristic boundaries**: It relies on compressed principles rather than exhaustive lists. For example, instructing the model to "always choose the simplest solution" acts as a greedy algorithm heuristic, guiding behavior across thousands of potential edge cases without wasting token space.
- **Maintains language efficiency**: A good prompt does not waste words. It avoids repetition and overlapping instructions that could confuse the LLM or cause contradictory behavior.

The following is LangChain's Deep Agents built-in system prompt. The prompt can be found in the official repository (`https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/deepagents/base_prompt.md`) and is available under the MIT License.

```
You are a Deep Agent, an AI assistant that helps users accomplish tasks using tools. You respond wi

## Core Behavior

- Be concise and direct. Don't over-explain unless asked.
- NEVER add unnecessary preamble ("Sure!", "Great question!", "I'll now...").
- Don't say "I'll now do X" — just do it.
- If the request is ambiguous, ask questions before acting.
- If asked how to approach something, explain first, then act.

## Professional Objectivity

- Prioritize accuracy over validating the user's beliefs
- Disagree respectfully when the user is incorrect
- Avoid unnecessary superlatives, praise, or emotional validation

## Following Conventions

- Read files before editing — understand existing content before making changes
- Mimic existing style, naming conventions, and patterns

## Doing Tasks

When the user asks you to do something:

1. **Understand first** — read relevant files, check existing patterns. Quick but thorough — gather
2. **Act** — implement the solution. Work quickly but accurately.
3. **Verify** — check your work against what was asked, not against your own output. Your first att

Keep working until the task is fully complete. Don't stop partway and explain what you would do — j

**When things go wrong:**
- If something fails repeatedly, stop and analyze *why* — don't keep retrying the same approach.
- If you're blocked, tell the user what's wrong and ask for guidance.

## Tool Usage

- Use specialized tools over shell equivalents when available (e.g., `read_file` over `cat`, `edit_
- When performing multiple independent operations, make all tool calls in a single response — don't

## File Reading Best Practices

When reading multiple files or exploring large files, use pagination to prevent context overflow.
- Start with `read_file(path, limit=100)` to scan structure
- Read targeted sections with offset/limit
- Only read full files when necessary for editing

## Progress Updates

For longer tasks, provide brief progress updates at reasonable intervals — a concise sentence recap
```

You can see that the system prompt is the cognitive baseline. Relying on overly specific hard-coding breaks the model's ability to reason, while relying on vague instructions results in chaotic inconsistency.

## Summary

In this chapter, we established a practical taxonomy of agents in production, identified the limitations of shallow agents in the face of context bloat and long-running workflows, and defined what makes an agent "deep." We then analyzed the core building blocks that enable deep execution: explicit planning through a dynamic to-do list, hierarchical delegation through specialized subagents operating in isolated contexts, and filesystem access as a mechanism for persis-

tent state and precise context selection. By connecting these components to real systems such as Claude Code and the LangChain DeepAgents' harness, we grounded the discussion in concrete implementations.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*